

Administration des Systèmes

MODULE 7: SYSTEMD

HÉDI NSIBI – FACULTÉ DES SCIENCES DE MONASTIR 2025 / 2026

Qu'est-ce que *systemd* ?

- **Définition:**

systemd est le **système d'initialisation et de gestion de services** utilisé par la majorité des distributions Linux modernes.
Il prend le rôle du processus **PID 1**, responsable du démarrage du système et de la supervision continue des services.

- **Fonctions principales**
- **Initialisation du système** au démarrage
- **Gestion et supervision des services** (start, stop, restart, auto-restart)
- **Gestion centralisée des journaux** via *journald*
- **Gestion des dépendances** entre services
- Gestion de l'arborescence des cgroups
- **Gestion des timers** (alternative moderne à cron)

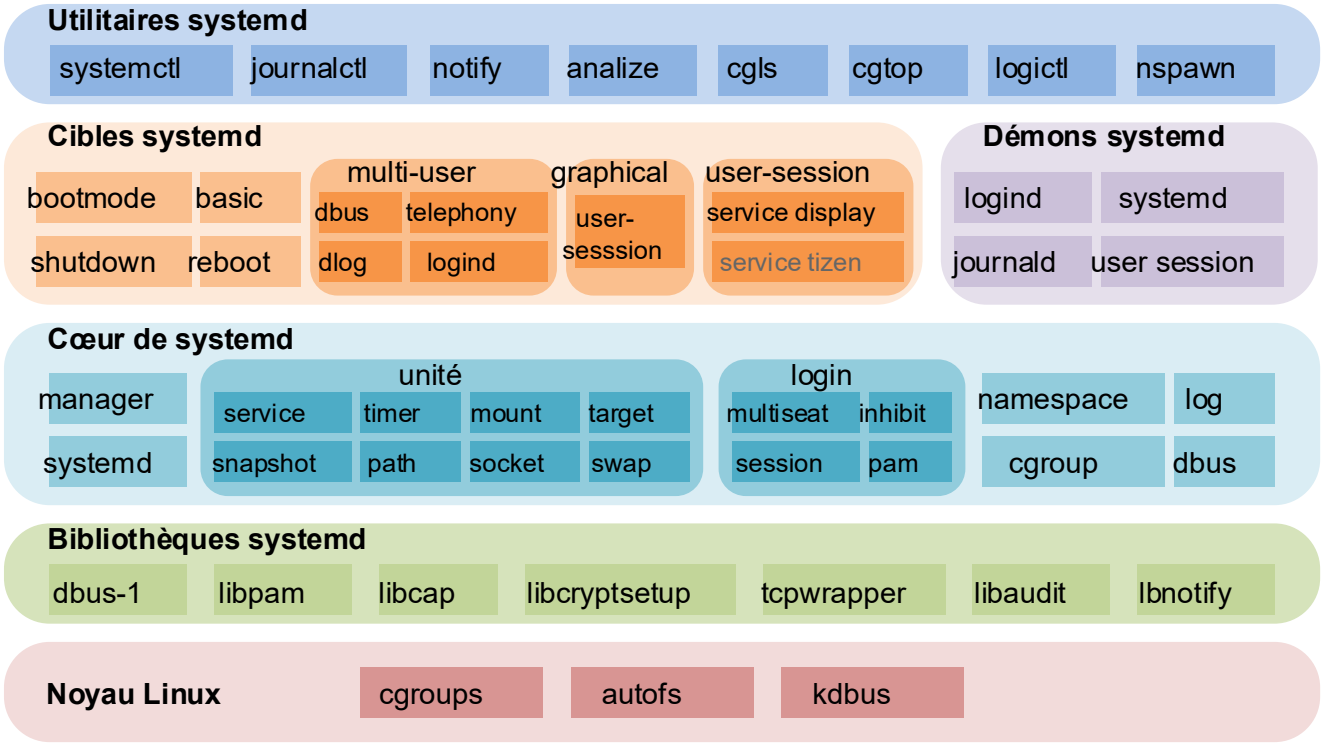
Distributions utilisant systemd

- ✓ **Ubuntu**
- ✓ **Debian**
- ✓ **Fedora**
- ✓ **CentOS / Rocky / AlmaLinux**
- ✓ **RHEL**
- ✓ **Arch Linux**
- ✓ **Objectif Général** : Offrir une plateforme unifiée, fiable et performante pour l'administration système moderne sur Linux.

Pourquoi systemd ?

- **Démarrage plus rapide**
  - Exécute plusieurs services en parallèle
  - Réduit le temps de boot
  - Optimise les dépendance entre services
- **Supervision automatique des services**
  - Relance automatique en cas d 'échec (Restart=)
  - Surveillance continue de l'état des processus
  - Gestion propre des arrêts /redémarrages
- **Standardisation de l'administration**
  - Commande unique : **systemctl**
  - Fichiers de configuration homogènes
  - Comportement identique sur toutes les distributions
- **Journalisation centralisée**
  - Journaux unifiés via **journald**
  - Recherche facile avec **journalctl**
  - Format structuré, filtrage avancé
- **Fonctionnalités modernes**
  - Timers
  - Activation par socket
  - Gestion intégrée des cgroups
  - Gestion avancée des dépendances

Architecture générale de systemd



Architecture générale de systemd

- **systemd (PID1)** le cœur du système :
  - Lance les services au démarrage
  - Supervise les processus
  - Gère les dépendances et les targets
- **systemd-journald** Service intégré de journalisation:
  - Collecte les logs du noyau
  - Centralise les logs des services
  - Permet un filtrage avancé via journalctl
- **systemd-logind** Responsable de la gestion des sessions :
  - Connexions utilisateurs
  - Gestion des TTY
  - Gestion de l'alimentation(verrouillage, veille…)
- **systemd-networkd (optionnel selon la distribution)** Gestion réseau intégrée:
  - Interfaces réseau
  - DHCP / statiques
  - Configuration simple via fichiers
- **systemd-cgroups** coordonne l'isolation via les control groups
  - Limitation de ressources CPU / RAM
  - Isolation de services
  - Hiérarchisation des ressources
- **Outils complémentaires**
  - **systemd-tmpfiles**: gestion des fichiers temporaires
  - **systemd-timedated:** gestion de l'heure
  - **systemd-localed:** locales système
  - **systemd-hotnamed**: informations machine

Vision d'ensemble

- systemd n'est pas seulement un service d'amorçage : c'est un **écosystème complet** pour piloter le système d'exploitation de manière cohérente et centralisée.

Les Units systemd

- Une ***unit*** = un objet géré par systemd.

Chaque unité représente une ressource ou un composant: service, timer, socket …

- **Principaux types :**

Type	Rôle	Exemples
.service	Gestion les deamons et proceesus	nginx.service, ssh.service
.socket	Activation par socket	cups.socket
.timer	Tâches planifiées	backup.timer
.mount	Montage de FS	home.mount
.device	Périphériques	dev-sda1.device
.target	Groupes d'unités / états	multi-user.target
.path	Déclenchement basé sur des changements de fichiers	myapp.path
.slice	Groupes de cgroups	user.slice

- Les fichiers unit se trouvent dans

- **/lib/systemd/system/ units du system**
- **/etc/systemd/system/ units personnalisées**

Commandes de base 1/4

- systemctl **— Gestion des services et Units**

Commande principale pour interagir avec systemd:

- **Démarrer**: systemctl start nom.service
- **Arrêter**: systemctl stop nom.service
- **Activer au démarrage**: systemctl enable nom.service
- **Statut**: systemctl status nom.service
- **Lister les units**: systemctl list-units

Commandes de base 2/4

- journalctl **— Consultation des journaux**

Permet d'afficher et filtrer les logs du système:

- **Touts les journaux**: journalctl
- **Journaux d'un service**: journalctl -u nom.service
- **Dernier boot**: journalctl –b
- **En temps réel**: journalctl -f

Commandes de base 3/4

- hostnamectl **— Informations machine**

Affiche et modifie l'identité du système :

- **Voir le hostname**: hostnamectl
- **Définir un nouveau hostname:** sudo hostnamectl set-hostname nouveau-nom
- Affiche aussi la version d'OS, architecture, etc.

Commandes de base 4/4

- localectl **— Gestion des locales du système**

Utilisé pour configurer la langue, le clavier, l'encodage:

- **Affiche la configuration**: localectl status
- **Changer la locale système** : sudo localectl set-locale LANG=fr_FR.UTF-8
- **Changer la disposition clavier** : sudo localectl set-keymap fr

Identifier systemd

- Vérifier quel processus est PID 1

Le PID 1 est le **premier processus lancé par le noyau** au démarrage. S'il s'agit de systemd, votre système utilise bien ce gestionnaire.

- **Commande :** ps -p 1 -o comm=
- **Sortie attendue :** systemd
- **Interprétation:**

- Si la sortie est **systemd**, vous êtes sur un système moderne utilisant systemd.
- Si la sortie est **init**, vous êtes sur un système basé sur **SysVinit** (rare aujourd'hui).

Gérer l'état global du système 1/2

1. Afficher l'état global du système

Permet de voir :

- Le target actif
- Les services actifs, inactifs ou en échec
- Les erreurs éventuelles

Commande : systemctl status

2. Lister les units actives

Affiche les unités actuellement chargées et en cours d'exécution.

Commande : systemctl list-units

Options utiles :

- --type=service → filtrer par type
- --state=failed → afficher uniquement les services en échec

Gérer l'état global du système 2/2

3. Lister *toutes* les units disponibles (actives ou non)

Liste les fichiers Units connus par systemd, même désactivés.

- **Commande :** systemctl list-unit-files

Indique pour chaque unité:

- Enabled
- Disabled
- Static
- Masked

Exemple .device dans systemd

- Les unités .device représentent les périphériques matériels détectés par le noyau (ex : disques, partitions, USB…).

- **Exemple 1 : voir toutes les unités device**

systemctl list-units --type=device

- **Exemple 2 : afficher une unité device spécifique**

systemctl status dev-sda.device

- **Exemple 3 : une clé USB montée**

systemctl status dev-sdb1.device

Statut d'une Unit

- **Exemple pratique :**

- systemctl status ssh
- systemctl status apache2
- systemctl status dev-sda.device

- **Etat possible:**

État	Signification
active	Le service ou le device fonctionne normalement
inactive	Le service est arrêté ou le device n'est pas utilisé
failed	Le service a rencontré une erreur dans l'exécution

Cycle de vie d'un service (systemd)

- **load** → systemd lit le fichier .service (configuration).
- **start** → lancement du service.
- **supervise** → systemd surveille le service (restart automatique, logs…).
- **reload** → recharge la configuration sans redémarrer (si supporté).
- **stop** → arrêt du service.
- **disable** → le service ne se lance plus au démarrage.

➡️ Idéal pour expliquer comment un service vit, tourne et s'arrête dans systemd.

Dépendances systemd

- Systemd permet de définir des relations entre unités via des directives.
- **Requires=**

- Dépendance **forte** :

Le service **ne peut pas démarrer** si l'unité référencée ne démarre pas.

Ex: Requires=network-online.target

- **Wants=**

- Dépendance **faible** :

Le service *tente* de démarrer l'unité liée, mais **ne dépend pas** de son succès.

Ex: Wants=sshd-keygen.service

- **Before=**

- Indique que le service doit être démarré **avant** une autre unité.

- **After=**

Indique que le service doit démarrer **après** une autre unité.
⚠️ Cela **n'ajoute pas de dépendance**, seulement une *ordre de démarrage*.

Que sont les Targets ?

- Définition

Les **targets** sont des états prédéfinis du système gérés par *systemd*.

- Rôle des targets:

- Ils jouent le rôle moderne des **runlevels** de SysVinit.
- Ils définissent **quelles unités** (services, sockets, devices, mounts…) doivent être actives pour un état donné.
- Permettent de passer rapidement d'un mode de fonctionnement à un autre.

➡️ Un *target* = un **ensemble d'unités** décrivant un **état global du système**.

Principaux Targets

Target	Rôle / Description
multi-user.target	Mode multi-utilisateur sans interface graphique (équivalent runlevel 3)
graphical.target	Démarrage avec serveur graphique + gestionnaire d'affichage
rescue.target	Mode maintenance avec root shell et services minimum
emergency.target	Shell root ultra minimal, pas de services ; utilisé pour réparations critiques

Voir le target actuel

- **Commande**

systemctl get-default

- **Exemple de sortie**

graphical.target

- **Utilité**

- Permet d'identifier le mode de démarrage par défaut
- Aide au diagnostic après changements ou pannes systèmes

Changer le target par défaut

- Modifier le mode de démarrage permanent

Pour définir un nouveau default target (affecte tous les prochains boots) :

sudo systemctl set-default multi-user.target

Explication:

- Change l'état du système au prochain redémarrage
- N'affecte **pas l'état actuel**
- Utile pour :
  - passer en mode console par défaut
  - désactiver l'interface graphique
  - forcer un serveur à démarrer en mode minimal

⚠️ **Attention**: Passer en rescue.target ou emergency.target peut couper SSH et services réseau.

Redémarrage et arrêt du système

- **Redémarrer**

systemctl reboot

- **Éteindre la machine**

systemctl poweroff

- **Notes importantes:**

- Ces commandes utilisent les appels systemd (meilleure gestion du shutdown)
- Les services sont arrêtés proprement, selon leurs dépendances
- Alternative pour redémarrage immédiat forcé :

systemctl reboot -i (à utiliser avec prudence)

Gestion des services — Commandes essentielles

- Commandes système les plus utilisées

- **start** → démarrer un service

systemctl start <service>

- **stop** → arrêter un service

systemctl stop <service>

- **restart** → arrêter puis relancer

systemctl restart <service>

- **reload** → recharger la configuration (si supporté)

systemctl reload <service>

- **enable** → activer au démarrage

systemctl enable <service>

- **disable** → désactiver au démarrage

systemctl disable <service>

- **status** → afficher état, logs, PID

systemctl status <service>

— Exemple : démarrer un service

- Lancer un service et l'activer au démarrage

systemctl start nginx

systemctl enable nginx

- start → démarre immédiatement le serveur web Nginx
- enable → crée un lien dans /etc/systemd/system/multi-user.target.wants/

→ Nginx démarre automatiquement au prochain reboot

— Analyse des fichiers Service

- Structure générale d'un fichier .service

[Unit]

Description=Description du service

After=network.target

[Service]

Type=simple

ExecStart=/usr/bin/script.sh

Restart=on-failure

[Install]

WantedBy=multi-user.target

- **Role des sections**

[Unit] → métadonnées, dépendances

[Service] → configuration du processus (comment lancer, gérer, redémarrer…)

[Install] → indique comment activer le service (enable)

— Paramètres importants

- **ExecStart=**

Commande exécutée pour démarrer le service

- **ExecReload=**

Commande pour recharger la configuration (si possible)

- **ExecStop=**

Commande pour arrêter proprement

- **Restart=**

Politique de redémarrage

Exemples: on-failure, always, no

- **Type=**

Type de service, parmi :

- simple(par défaut) — le service reste au premier processus lancé
- Forking — utilisé pour les démons qui se détachent en arrière-plan
- Oneshot pour exécuter une tâche puis sortir (scripts initiaux)

— Service personnalisé

- **Exemple minimal d'un service systemd exécutant un script shell**
- Fichier : /etc/systemd/system/backup.service

[Unit]

Description=Script de sauvegarde personnalisé

[Service]

Type=oneshot

ExecStart=/usr/local/bin/backup.sh

[Install]

WantedBy=multi-user.target

- **Activation du service:**

sudo systemctl daemon-reload

sudo systemctl start backup.service

sudo systemctl enable backup.service

- **A retenir**

- Type=oneshot car le script s'exécute puis se termine
- Nécessite un daemon-reload après création ou modification
- Très utile pour automatiser des tâches administratives

— Recharger systemd

- **Pourquoi recharger systemd ?**

Lorsque tu crées ou modifies :

- un fichier **.service**
- un fichier **.timer**
- ou toute unité systemd

— systemd ne détecte pas automatiquement les changements.

- **Commande indispensable**

sudo systemctl daemon-reload

- **Effet:**

- Recharge toutes les définitions d'unités
- Met à jour les dépendances
- Ne redémarre **aucun service**

- **À retenir**

Toujours exécuter daemon-reload après avoir créé ou modifier un fichier dans /etc/systemd/system/ ou /usr/lib/systemd/system/

— Dépannage des services 1/2

- **Outils essentiels pour diagnostiquer un service**

Afficher les logs d'un service

journalctl -u nom.service

- Affiche les derniers événements et erreurs
- Options utiles :
  - -f pour suivre les logs en temps réel
  - --since « 10 min ago » pour filtrer

- **Vérifier l'état complet**

systemctl status nom.service

— Dépannage des services 2/2

- **Vérifier l'état complet**

systemctl status nom.service

Montre :

- état (active / inactive / failed)
- PID
- journal récent
- code d'erreur éventuel (exit code)

- **Réinitialiser l'état « failed »**

systemctl reset-failed

- **Pourquoi l'utiliser ?**

- Supprime les marqueurs d'échec persistants
- Utile après un service qui a crashé mais qui fonctionne maintenant
- Permet un diagnostic plus propre

Gestion des Timers — Pourquoi les timers ?

- **Pourquoi utiliser les *timers systemd* plutôt que *cron* ?**

✓ **Remplacent cron de manière moderne**

Les timers offrent un solution native pour planifier des tâches, sans dépendre de crontab

✓ **Logs intégrés automatiquement**

Chaque exécution est enregistrée dans journalctl, ce qui facilite :

- La traçabilité
- L'audit
- Le dépannage

✓ **Dépendances systemd**

Les Timers peuvent dépendre de targets, services, sockets…

→ Permet de lancer une tâche seulement si le système est prêt

✓ **Supervision automatique**

Gestion des erreurs, relances automatiques, contrôle d'exit codes, protection de ressources (cgroups)

- **À retenir**

— Les timers apportent robustesse, supervision, logs et intégration complète dans systemd.

— Anatomie d'un Timer 1/2

- Les timers systemd se composent **obligatoirement de deux fichiers** :
- **Fichier .service**

Définit **ce qui doit être exécuté**.

Exemple : script, commande, programme, sauvegarde…

Structure classique :

[Service]

ExecStart=/usr/local/bin/backup.sh

— Anatomie d'un Timer 2/2

- Fichier .timer

Définit quand le service doit s'exécuter

Exemple de base

[Timer]

OnCalendar=daily

- **Fonctionnement**

Le .timer déclenche le .service

Les deux fichiers doivent porter **le même nom** :

- backup.service
- backup.timer

- **Avantage**

Séparation claire entre:

- la logique (service)
- la planification (timer)

Exemple d'un Timer

- **Exemple complet : *backup.service* + *backup.timer***
- **Fichier:** /etc/systemd/system/backup.service

[Unit]

Description=Script de sauvegarde automatique

[Service]

Type=oneshot

ExecStart=/usr/local/bin/backup.sh

- Type=oneshot : exécution unique, pas de processus permanent
- Le script backup.sh contient la logique de sauvegarde

Exemple d'un Timer 2

- **Fichier** : /etc/systemd/system/backup.timer

[Unit]

Description=Timer pour la sauvegarde quotidienne

[Timer]

OnCalendar=daily

Persistent=true

[Install]

WantedBy=timers.target

- OnCalendar=daily → exécution chaque jour
- Persistent=true → exécute la tâche si elle a été manquée (machine éteinte)

— Activer et vérifier

- **Activer le timer immédiatement**

systemctl enable --now backup.timer

Effet:

- Enable → activation permanente au boot
- --now → lancement immédiat sans attendre

Voir les timers actifs

systemctl list-timers

- **Afiiche la date de dernière exécution**
- **Affiche date de prochaine exécution**
- **Affiche unité .timer associée**
- **Affiche .service correspondante**

— Types de déclenchement

- Paramètres utilisés pour définir *quand* exécuter une tâche
- **OnBootSec=**

Temps d'attente **après le démarrage** avant d'exécuter le service.

- **OnActiveSec=**

Temps après l'activation du timer.

- **OnCalendat=**

Planification basée sur le calendrier (comme cron, mais plus puissante).

Messages journaux — Commandes journald

- **Commande générale**

Journalctl

Affiche tous les logs du système collectés par journald

Inclut: services, kernel, boot, erreurs, informations...

- **Logs d'un service spécifique**

journalctl –u ssh

Filtre uniquement les logs du service ssh.service

Trés utile pour diagnostiquer un service système

- **Logs du dernier démarrage**

Journalctl –b

- -b = boot
- Montre tous les messages depuis le dernier démarrage
- Option utile:: journalctl –b –1 → démarrage précédent

— Filtrage avancé

Filtrer par période

- **Depuis une heure/ un momdent précis**

journalctl --since "1 hour ago"

journalctl --since "2025-02-18 08:00"

- **Jusqu'à un moment**

journalctl --until "10 minutes ago"

Filtrer par priorité (sévérité)

journalctl -p err

Suivi en temps réel

journalctl -f

— Gestion de l'espace des logs

- **Nettoyer automatiquement les journaux**
- **Limiter la taille totale des logs :**

sudo journalctl --vacuum-size=500M

→ Conserve uniquement les 500 Mo les plus récents.

Limiter par date :

sudo journalctl --vacuum-time=7d

→ Supprime les journaux plus vieux que 7 jours.

Limiter le nombre de fichiers :

sudo journalctl --vacuum-files=10